

ChatGPT - CH DRL



可以告訴我要怎麼完成這個作業嗎 需要繳交哪些東西

可以。這份作業主要是要你從基礎 DQN 開始，逐步加入 DQN 改良版，最後在更困難的 random mode 下測試穩定性。

作業整體目標

你要完成三個主要部分：

題目	模式	要做什麼	分數
HW3-1	static mode	跑 Naive DQN 或 Experience Replay DQN，並寫理解報告	30%
HW3-2	player mode	實作並比較 Double DQN、Dueling DQN	40%
HW3-3	random mode	將 DQN 改寫成 Keras 或 PyTorch Lightning，並加入訓練技巧	30%
HW3-4	random mode	Rainbow DQN，應該是加分題	加分

你需要繳交哪些東西？

建議你最後繳交一個資料夾，內容如下：

```
HW3_DQN/  
|  
├─ report.pdf 或 report.docx  
|  
├─ hw3_1_static_dqn.py  
└─ hw3_2_double_dqn.py
```

```
├── hw3_2_dueling_dqn.py
├── hw3_3_random_lightning.py 或 hw3_3_random_keras.py
├──
├── results/
│   ├── hw3_1_loss.png
│   ├── hw3_1_reward.png
│   ├── hw3_2_comparison_reward.png
│   ├── hw3_2_comparison_loss.png
│   ├── hw3_3_random_reward.png
│   └── hw3_3_random_loss.png
├──
├── models/
│   ├── static_dqn.pth
│   ├── double_dqn.pth
│   ├── dueling_dqn.pth
│   └── random_dqn.pth
├──
└── README.md
```

如果老師沒有要求交 model，可以不用交 `.pth`，但建議保留。

HW3-1：Naive DQN for static mode

你要做什麼？

這題是最基本的 DQN，環境是 **static mode**。

也就是：

物件	位置
Player	固定在 <code>(0, 3)</code>
Goal	固定在 <code>(0, 0)</code>
Pit	固定在 <code>(0, 1)</code>
Wall	固定在 <code>(1, 1)</code>

這個模式最簡單，因為每次遊戲的初始狀態都一樣，所以比較容易訓練成功。

你需要完成的內容

你可以選擇：

1. 跑 Naive DQN

2. 或跑 Experience Replay DQN

但建議你用 **Experience Replay DQN**，因為比較穩定，也比較像正式 DQN。

這題報告要寫什麼？

你要寫一份短的 understanding report，內容可以包含：

1. DQN 是什麼？

DQN 是用神經網路來近似 Q-table。

原本 Q-learning 是：

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

DQN 則是讓神經網路輸入 state，輸出每個 action 的 Q-value。

例如：

state → neural network → [Q(up), Q(down), Q(left), Q(right)]

2. Experience Replay 是什麼？

Experience Replay Buffer 會儲存過去的經驗：

(state, action, reward, next_state, done)

訓練時不是只用最新的一筆資料，而是從 buffer 裡隨機抽一批資料來訓練。

好處是：

可以打亂資料相關性，讓訓練比較穩定。

3. 你跑出來的結果

你至少要放：

圖	說明
loss curve	看模型是否逐漸收斂
reward curve	看 agent 是否越來越會

圖	說明
	走到 goal
epsilon curve	看探索率是否下降
測試結果	訓練後是否能成功走到 goal

HW3-2：Enhanced DQN Variants for player mode

這題是 40%，最重要。

你要做什麼？

你要在 **player mode** 下實作並比較：

- 1. Double DQN
- 2. Dueling DQN

player mode 的意思是：

物件	位置
Player	隨機位置
Goal	固定
Pit	固定
Wall	固定

所以它比 static mode 困難，因為玩家每次起始位置不同，模型要學會泛化。

Double DQN 要做什麼？

基本 DQN 的問題

普通 DQN 在更新 target Q-value 時會用：

$$r + \gamma \max_a Q(s', a)$$

問題是它很容易高估 Q-value。

Double DQN 的改法

Double DQN 用兩個網路分工：

網路	功能
online network	選擇下一步 action
target network	計算該 action 的 Q-value

更新方式概念如下：

```
</> Python
next_action = online_net(next_state).argmax(dim=1)
next_q = target_net(next_state).gather(1, next_action)
target = reward + gamma * next_q
```

你在報告可以寫：

Double DQN 可以降低 Q-value overestimation，使訓練結果更穩定。

Dueling DQN 要做什麼？

Dueling DQN 是改網路架構。

它不是直接輸出 Q-value，而是拆成兩條分支：

分支	意義
Value stream	評估目前 state 好不好
Advantage stream	評估每個 action 比其他 action 好多少

公式是：

$$Q(s,a) = V(s) + A(s,a) - mean(A(s,a))$$

概念是：

有些狀態下，不管往哪個方向走都差不多。
Dueling DQN 可以先學這個 state 本身的價值，再判斷 action 的差異。

HW3-2 報告要比較什麼？

你要比較至少三個模型：

方法	是否要比較
Basic DQN	建議保留當 baseline
Double DQN	必須
Dueling DQN	必須

比較指標可以放：

指標	說明
average reward	平均回饋是否變高
loss curve	訓練是否穩定
success rate	測試時成功到達 goal 的比例
steps to goal	平均幾步到達 goal
convergence speed	哪一個比較快收斂

HW3-3：Enhance DQN for random mode with training tips

這題是 random mode，難度最高。

random mode 是什麼？

物件	位置
Player	隨機
Goal	隨機
Pit	隨機
Wall	隨機

所以每次環境都不同，模型不能只記固定路線，必須真的學會策略。

你要做什麼？

題目要求你把 DQN model 從 PyTorch 轉成下面其中一個：

- 1. Keras
- 2. PyTorch Lightning

如果你原本就是用 PyTorch，建議選 **PyTorch Lightning**，因為比較接近原本程式，不用整個重寫。

你可以加入哪些 training tips？

作業提到可以加訓練技巧，例如：

技巧	功能
Experience Replay	穩定訓練
Target Network	避免 target 一直變動
Gradient Clipping	避免梯度爆炸
Learning Rate Scheduler	動態調整學習率
Epsilon Decay	由探索慢慢轉向利用
Double DQN update	降低 Q-value 高估
Reward smoothing	讓 reward curve 比較好分析

最推薦你至少加這三個：

- 1. Gradient clipping
 - 2. Learning rate scheduler
 - 3. Target network update

這樣報告比較好寫，也符合題目要求。

HW3-4：Rainbow DQN 加分題

這是加分題，不一定要做。

Rainbow DQN 是把很多 DQN 改良技巧整合起來，例如：

技術	功能
Double DQN	降低 Q-value overestimation
Dueling DQN	改善 state-value 學習
Prioritized Replay	重要經驗優先抽樣
Multi-step Learning	使用多步 reward
NoisyNet	改善 exploration
Distributional DQN	學習 reward 分布

如果時間不夠，你可以只做前三個：

Double DQN + Dueling DQN + Prioritized Replay

然後在報告寫這是 simplified Rainbow DQN。

最後報告建議架構

你的 report 可以這樣寫：

1. Introduction
 - 本作業目的
 - DQN 與 Q-learning 的關係

2. Environment Description
 - static mode
 - player mode
 - random mode

3. HW3-1: Naive DQN / Experience Replay DQN
 - 方法說明
 - 程式架構
 - 訓練結果
 - loss / reward 分析

4. HW3-2: Double DQN and Dueling DQN
 - Double DQN 原理
 - Dueling DQN 原理
 - 與 Basic DQN 比較
 - 結果圖與表格

5. HW3-3: Random Mode with Training Tips
- PyTorch Lightning 或 Keras 架構

- 加入哪些 training techniques

- 訓練結果

- 穩定性分析
6. Discussion
- 哪個方法收斂最快

- 哪個方法最穩定

- random mode 為什麼比較難
7. Conclusion
- 總結三題結果

建議你最後一定要有的圖

至少準備這些圖：

圖名	對應題目
HW3-1 reward curve	static DQN 是否學會
HW3-1 loss curve	loss 是否下降
HW3-2 reward comparison	Basic DQN vs Double DQN vs Dueling DQN
HW3-2 loss comparison	三種方法穩定性比較
HW3-3 random mode reward curve	random mode 訓練效果
HW3-3 random mode loss curve	訓練是否穩定

你可以怎麼分配實作順序？

建議照這個順序做：

- Step 1：先跑老師給的 starter code，確認 static mode 可以跑
- Step 2：加入 Experience Replay Buffer
- Step 3：畫 reward / loss curve
- Step 4：改成 player mode

- Step 5：實作 Double DQN
- Step 6：實作 Dueling DQN
- Step 7：比較 Basic DQN、Double DQN、Dueling DQN
- Step 8：改成 random mode
- Step 9：改寫成 PyTorch Lightning 或 Keras
- Step 10：加入 gradient clipping / scheduler / target update
- Step 11：整理報告

最簡單可交版本

如果你想先求穩，可以這樣完成：

部分	最低可交內容
HW3-1	Experience Replay DQN + static mode 結果
HW3-2	Double DQN、Dueling DQN 各跑一次，和 Basic DQN 比較
HW3-3	PyTorch Lightning 版本 + 加 gradient clipping 和 scheduler
Report	放方法說明、結果圖、比較表、心得

你報告裡可以寫的重點結論

可以先用這種方向：

在 static mode 中，由於 Player、Goal、Pit、Wall 的位置皆固定，因此 Basic DQN 或 Exper

在 player mode 中，Player 的起始位置隨機，使狀態空間變大，因此 Basic DQN 的訓練波動較明顯

在 random mode 中，所有物件位置皆隨機，環境變化最大，因此訓練最不穩定。透過 gradient clip

你這份作業最重要的是：

不要只交程式，要交「比較結果」和「你理解為什麼這些 DQN variants 有幫助」。



Pasted code.py

Python

可以講解3-1嗎

可以，HW3-1 這題重點是：在 **static mode** 的 **GridWorld** 裡，先完成一個基本 **DQN**，並說明你理解 **DQN** 與 **Experience Replay** 的作用。你目前貼的程式已經是「Basic DQN + Experience Replay Buffer」版本，不是完全 Naive DQN。

HW3-1 題目在要你做什麼？

題目寫的是：

Naive DQN for static mode
Run the provided code naive or Experience buffer replay
Submit a short understanding report

意思是你要在 **static mode** 下訓練 DQN。

static mode 是最簡單的情況：

物件	位置
Player	固定
Goal	固定
Pit	固定
Wall	固定

也就是每一場遊戲的地圖都一樣，所以 **agent** 只要學會一條固定路線，就可以到達 **Goal**。

你的程式在做什麼？

你的程式主要分成 5 個部分：

1. Action Mapping
2. Replay Buffer
3. DQN Model
4. Training
5. Testing / Visualization

1. Action Mapping

你的程式這裡：

Python

```
action_set = {0: 'u', 1: 'd', 2: 'l', 3: 'r'}
```

意思是把神經網路輸出的 action index 轉成 GridWorld 可以接受的移動指令。

index	動作
0	up
1	down
2	left
3	right

DQN 輸出會是 4 個 Q-value：

```
[Q(up), Q(down), Q(left), Q(right)]
```

然後選 Q-value 最大的動作。

2. Replay Buffer 是什麼？

你的程式有這段：

Python

```
class ReplayBuffer:
    def __init__(self, capacity=1000):
        self.buffer = deque(maxlen=capacity)
```

Replay Buffer 是用來存過去經驗的地方。

每一筆經驗長這樣：

```
(state, action, reward, next_state, done)
```

也就是：

名稱	意思
state	目前狀態

名稱 action	意思 這一步做了什麼動作
reward	得到多少獎勵
next_state	做完動作後的新狀態
done	是否遊戲結束

為什麼要用 Replay Buffer ？

如果不用 Replay Buffer，DQN 會拿「剛剛發生的資料」直接訓練。

問題是 GridWorld 的資料順序很相關，例如：

往上 → 往上 → 往右 → 往右

這樣連續資料太相似，訓練容易不穩定。

Replay Buffer 的做法是：

Python

```
batch = random.sample(self.buffer, batch_size)
```

也就是從過去經驗中**隨機抽一批資料**來訓練。

好處是：

打散資料順序，降低資料相關性，讓 DQN 訓練比較穩定。

3. DQN Model 是什麼？

你的模型是：

Python

```
self.fc1 = nn.Linear(64, 150)
self.fc2 = nn.Linear(150, 100)
self.fc3 = nn.Linear(100, 4)
```

因為 GridWorld 是 4×4，而且每個格子用 4 個 channel 表示，所以輸入大小是：

$4 \times 4 \times 4 = 64$

輸出是 4，代表四個動作的 Q-value：

up, down, left, right

所以整個網路可以理解成：

目前地圖狀態 → DQN → 四個動作的分數

例如輸出：

[0.2, -0.5, 1.3, 0.7]

代表：

動作	Q-value
up	0.2
down	-0.5
left	1.3
right	0.7

模型會選 Q-value 最大的：

left

4. get_state(env) 在做什麼？

你的程式：

`</> Python`

```
state = env.board.render_np().reshape(1, 64)
return state + np.random.rand(1, 64)/100.0
```

這裡把 GridWorld 的棋盤轉成神經網路可以吃的向量。

原本地圖是 4×4×4，展平成：

1 × 64

後面加一點很小的 noise：

```
</> Python
```

```
np.random.rand(1, 64)/100.0
```

這是參考書上的做法，目的是避免完全相同的 state 讓模型訓練太死板。

你報告可以寫：

The state is represented as a 64-dimensional vector converted from the 4×4×4 Gr

中文可以寫：

本作業將 4×4×4 的 GridWorld 狀態展平成 64 維向量，作為 DQN 的輸入。同時加入微小隨機雜訊，

5. Epsilon-greedy 是什麼？

你的程式：

```
</> Python
```

```
if random.random() < epsilon:
    action_ = np.random.randint(0, 4)
else:
    action_ = torch.argmax(qval).item()
```

這叫做 **epsilon-greedy policy**。

意思是：

情況	行為
random.random() < epsilon	隨機探索
random.random() ≥ epsilon	選 Q-value 最大的動作

一開始：

```
</> Python
```

```
epsilon = 1.0
```

代表 agent 幾乎都在亂走。

後面 epsilon 會慢慢下降：

</> Python

```
if epsilon > 0.1:
    epsilon -= (1/epochs)
```

代表訓練越久，agent 越相信自己學到的 Q-value。

你可以這樣解釋：

訓練初期使用較高的 epsilon 讓 agent 探索不同路徑；隨著訓練進行，epsilon 逐漸下降，使 agent

6. DQN 的更新公式

你的程式這段是最重要的：

</> Python

```
Y = r_batch + gamma * ((1 - done_batch) * torch.max(qval_batch, dim=1)[0])
X = qval_batch.gather(dim=1, index=a_batch.unsqueeze(1)).squeeze(1)
loss = loss_fn(X, Y)
```

這裡對應到 DQN 的 target：

$$Y = r + \gamma \max_{a'} Q(s', a')$$

如果 done = True，代表遊戲已經結束，就不看下一個 state，所以變成：

$$Y = r$$

你的程式用：

</> Python

```
(1 - done_batch)
```

來處理這件事。

done	意思	target
0	遊戲還沒結束	$r + \gamma \max Q(s', a')$
1	遊戲結束	r

7. X 是什麼？Y 是什麼？

這裡很多人會搞混。

X：模型目前預測的 Q-value

Python

```
X = qval_batch.gather(dim=1, index=a_batch.unsqueeze(1)).squeeze(1)
```

DQN 對每個 state 會輸出 4 個 Q-value。

例如：

```
Q(s) = [0.1, 0.4, -0.2, 0.9]
```

假設這次實際做的 action 是 right，也就是 index = 3。

那我們只要拿：

```
Q(s, right) = 0.9
```

所以 `gather()` 的功能就是：

從四個 Q-value 裡面，取出當初實際採取的那個 action 對應的 Q-value。

Y：DQN target

Python

```
Y = r + gamma * max Q(next_state)
```

Y 是目標值，也就是我們希望模型學到的值。

最後用 `MSELoss`：

Python

```
loss = loss_fn(X, Y)
```

意思是讓模型的預測值 X 越來越接近 target Y。

8. 你的程式目前有一個重要地方建議修改

你的 `train()` 裡面一開始只建立一次環境：

</> Python

```
env = Gridworld(size=4, mode='static')
```

但是每個 epoch 開始時沒有 reset 環境。

你目前是：

</> Python

```
for i in range(epochs):  
    state = get_state(env)
```

建議改成：

</> Python

```
for i in range(epochs):  
    env.initGridStatic()  
    state = get_state(env)
```

或是：

</> Python

```
for i in range(epochs):  
    env = Gridworld(size=4, mode='static')  
    state = get_state(env)
```

比較推薦第一種。

因為每一個 episode 開始時，都應該重新初始化 GridWorld。

9. 建議你修改後的 train 開頭

把這段：

</> Python

```
for i in range(epochs):  
    state = get_state(env)  
    done = False  
    status = 1  
    ep_reward = 0
```

改成：

</> Python

```
for i in range(epochs):  
    env.initGridStatic()
```

```
state = get_state(env)
done = False
status = 1
ep_reward = 0
```

這樣每一局遊戲都會從 static mode 的固定初始狀態重新開始。

10. 另一個建議：a_batch 要轉成 long

你之前遇到過這個錯誤：

```
RuntimeError: gather(): Expected dtype int64 for index
```

所以這裡：

</> Python

```
a_batch = torch.from_numpy(a_batch)
```

建議改成：

</> Python

```
a_batch = torch.from_numpy(a_batch).long()
```

因為 `gather()` 的 index 必須是 `int64`。

也就是這段要改成：

</> Python

```
a_batch = torch.from_numpy(a_batch).long()
```

這樣在 Windows 或 Linux 都比較穩。

11. HW3-1 最後要看什麼結果？

你要看兩張圖：

```
loss_curve_3_1.png
reward_curve_3_1.png
```

你的程式已經有存圖。

Loss curve 怎麼看？

loss 不一定會一路下降，因為 DQN 的 target 會一直變，而且資料來自 replay buffer。

正常情況可能是：

前期 loss 波動大
中後期慢慢變小或維持在較低範圍
偶爾會突然上升

所以 DQN 的 loss curve 不像一般 supervised learning 那麼漂亮。

Reward curve 怎麼看？

reward 比 loss 更重要。

在 static mode 中，如果訓練成功，reward 應該會越來越常接近：

+10

代表 agent 成功走到 Goal。

如果 reward 常常是：

-10

代表 agent 常掉進 Pit。

如果 reward 一直很低或亂跳，代表還沒學好。

12. Testing 結果怎麼判斷？

你的 `test()` 會輸出：

Win Rate: xx.x%, Avg Steps: xx.x

這兩個最重要。

指標	意義
Win Rate	成功到達 Goal 的比例
Avg Steps	平均幾步到達 Goal 或結束

static mode 通常應該可以訓練到很高的 win rate。

理想結果：

Win Rate 接近 100%
Avg Steps 越少越好

13. HW3-1 報告可以怎麼寫？

你可以照這個結構寫：

1. Environment
本實驗使用 static mode GridWorld。Player、Goal、Pit、Wall 的位置皆固定，因此此模式
2. DQN Architecture
本實驗使用三層 fully connected neural network。輸入為 64 維狀態向量，輸出為四個 act
3. Experience Replay
Replay Buffer 用來儲存過去的 transition，包括 state、action、reward、next_state
4. Training Process
Agent 使用 epsilon-greedy policy 選擇行動。訓練初期 epsilon 較高，使 agent 進行探索
5. Result
從 reward curve 可觀察 agent 是否逐漸學會到達 goal；從 loss curve 可觀察 Q-value 的

14. 你這題可以交哪些東西？

HW3-1 建議交：

1. hw3_1_static_dqn.py
2. loss_curve_3_1.png
3. reward_curve_3_1.png
4. 測試結果截圖，例如 Win Rate 和 Avg Steps
5. 一小段 understanding report

如果老師要完整報告，HW3-1 的部分大概寫 1~2 頁就夠了。

15. 你可以在報告寫的結論

可以直接用這段改成自己的結果：

在 HW3-1 中，本實驗於 static mode GridWorld 中實作 DQN with Experience Replay。由於

你現在的 HW3-1 大方向是對的，最需要先改的兩個地方是：

`</>` Python

```
env.initGridStatic()  
a_batch = torch.from_numpy(a_batch).long()
```

改完後再跑，看 reward curve 和 win rate 是否正常。

3-3和3-4叫難收斂是因為是random的對嗎

對，3-3 和 3-4 會比較難收斂，主要原因就是 **random mode**，但不只是「random」這個字而已，而是它讓環境變得更不穩定、更難學。

為什麼 random mode 難收斂？

在前面的模式中，環境比較固定：

模式	Player	Goal	Pit	Wall	難度
static	固定	固定	固定	固定	最簡單
player	隨機	固定	固定	固定	中等
random	隨機	隨機	隨機	隨機	最難

3-3 為什麼難？

HW3-3 是 **random mode**。

也就是每一場遊戲中：

Player 位置會變
Goal 位置會變
Pit 位置會變
Wall 位置會變

所以 agent 不能只背一條路線。

在 static mode 裡，它可能只要學會：

從固定起點走到固定 Goal

可是 random mode 裡，它要學的是：

不管 Goal 在哪裡，都要找到 Goal

不管 Pit 在哪裡，都要避開 Pit

不管 Wall 在哪裡，都要繞過 Wall

這就變成真正的「策略學習」，所以比較難收斂。

為什麼 reward 會更不穩？

因為每一局地圖都不一樣。

有些局可能 Goal 很近：

很快得到 +10

有些局可能 Pit 很近：

很快得到 -10

有些局可能 Wall 擋住路：

走很多步才結束

所以 reward curve 會比較抖，不會像 static mode 那麼漂亮。

為什麼 loss 也會比較亂？

DQN 的 loss 是在學：

$$Q(s, a) \approx r + \gamma \max_{a'} Q(s', a')$$

但是 random mode 中，每次遇到的 state 差異很大。

同樣是「往右走」，在不同地圖可能結果完全不同：

狀況	往右走的結果
----	--------

右邊是 Goal	得到 +10
----------	--------

狀況	往右走的結果
右邊是 Pit	得到 -10
右邊是 Wall	可能卡住
右邊是空格	繼續走

所以模型比較難快速學出穩定的 Q-value 。

3-4 為什麼也難？

HW3-4 是 Rainbow DQN，用在 random mode 。

Rainbow DQN 理論上比普通 DQN 強，因為它結合很多技巧，例如：

- Double DQN
- Dueling DQN
- Prioritized Replay
- Multi-step Learning
- NoisyNet
- Distributional DQN

但是實作上它也比較難，因為：

1. 結構比較複雜
2. 超參數比較多
3. random mode 本身就不穩定
4. 每個技巧如果沒有調好，可能反而更難收斂

所以 3-4 不是因為 Rainbow DQN 本身差，而是因為：

Rainbow DQN + random mode + 多個超參數

會讓訓練更難調整。

報告可以這樣寫

你可以在報告裡寫這段：

HW3-3 and HW3-4 are more difficult to converge mainly because they are trained

中文可以寫：

HW3-3 與 HW3-4 較難收斂的主要原因在於它們使用 random mode。在 random mode 中，Player、

所以你的理解是對的：

3-3 和 3-4 比較難收斂，核心原因就是 random mode 讓環境變化太大。